Jeremy Barton
Term Project
DSC550-T302
Spring 2026

WigglNET: Niche Market Data Mining and Brokerage Service

Milestone 1: Data Selection and EDA

IMPORTANT NOTE: Project reframed to validate effectiveness of RoBERTa model rather than predict accuracy of its outputs. Ground-truth data needed for prediction, but not immediately available. In practice it will be collected through client outcomes, feedback and business results.

Business Pitch

"Data is the new oil" is often used to describe the immense monetary benefit of holding valuable information that can lead to winning critical business decisions. The world now demands *intelligent* data that thinks for itself. Data that will grow and learn as their company does, giving it a mind of its own.

Business Question

Small e-commerce sellers and product manufacturers struggle to understand what is popular, why, and what will be popular in the future. This data could be retrieved by a search engine or AI, but there is more value in using advanced models specially trained to identify people, organizations or entities related to a specific product category to discover these insights rather than a Large Language Model trained on everything that happens to have picked up data related to a particular target market.

A sentiment analysis model called RoBERTa, developed by Facebook, applies labels to keywords and entities based on perceived context. The reference data comes from various social media platforms, like Twitter.

While the RoBERTa model applies labels to keywords and entities that may be related to the target market, it cannot be treated as truth. This is because there are no "ground-truths" for the model to learn what is true and not true. Some could say this is a metaphor for civilization's loss of what we have, as a collective, previously believed in, but no longer do. In the same way, the decisions we make are based on predictions that were only supported by patterns, not principles.

In practice, ground truths to improve RoBERTa would be realized by client outcomes, manual review or business results. This would come after the client has received the report of keyword/entities that were labeled by RoBERTa. WigglNet would then work with the client to monitor outcomes based on the decisions made by RoBERTa - confirming or denying the labels produced by the model.

Therefore, the question being asked in this business problem must be: ***"Can we validate the sentiment labels produced by RoBERTa for a specific product market niche?"*** Doing so would give extra reassurance to the initial pattern-based report provided to clients - improving perceived trustworthiness.

Exploring the Data

A static CSV file contains the results of a query that pulls back combined article, keyword, entity and sentiment information by joining to four other tables on either ARTICLE_ID or KEYWORD_ID. Because this is time series data, the query can ignore all rows where PUBLISHED_DATE is NULL.

It includes the CONFIDENCE field from ARTICLE_ENTITY_SENTIMENT, which indicates the RoBERTa model's confidence level (0-1) in the sentiment classification for each entity.

In [6]:
```
# Imports and Processing
import sys
import pandas as pd
df = pd.read_csv('keywords.csv')
```

In [7]:
```
# Display basic info about the dataset
print(f"Dataset shape: {df.shape}")
print(f"\nColumn names: {df.columns.tolist()}")
```

```
Dataset shape: (19367, 9)

Column names: ['ARTICLE_ID', 'TITLE', 'PUBLISHED_DATE', 'KEYWORD_TEXT', 'ENTITY_TEXT', 'ENTITY_TYPE', 'SENTIMENT', '
CONFIDENCE', 'MENTIONS']
```

Predictor Variables

To validate RoBERTa sentiment labels, the model will use the following predictor variables.

- ENTITY_TYPE
- KEYWORD_TEXT
- ENTITY_TEXT
- TITLE

- PUBLISHED_DATE
- SOURCE

IMPORTANT NOTE: `ENTITY_TYPE` , `KEYWORD_TEXT` , and `ENTITY_TEXT` are produced by an upstream Named Entity Recognition (NER) model. NER will be evaluated with the same rigor as RoBERTa, but under a separate workstream. The current project is scoped exclusively to RoBERTa.

How many articles are in this dataset?

In [8]:
```
total_articles = df["ARTICLE_ID"].nunique()
print(f"There are {total_articles} articles in this dataset.")
```

```
There are 132 articles in this dataset.
```

If there are only 132 articles, why are there 19,637 rows?

This dataset includes information on metadata about keywords and entities in an article, not just a list of each article. Every keyword has an entity text, type, sentiment, and number of mentions.

What are the max and min published dates?

In [9]:
```
max_pub = df["PUBLISHED_DATE"].max()
print(f"Max published date is {max_pub}")

min_pub = df["PUBLISHED_DATE"].min()
print(f"Min published date is {min_pub}")
```

```
Max published date is 2026-04-10
Min published date is 2022-04-01
```

Identifying Entities with Positive or Negative Sentiment, Ordered by Negative Sentiment

This analysis focuses on entities with strong sentiment signals (positive or negative), excluding purely neutral entities, and orders them by negative sentiment to highlight problematic entities first. A client might use this graph to understand what brands or keywords to avoid in future business decisions.

Ordering by negative mentions (rather than total volume) surfaces entities that would otherwise be hidden under high-volume neutral coverage, making it the more useful view for a brokerage service flagging risk.

Data Preparation

In [10]:
```
# Import visualization libraries
import matplotlib.pyplot as plt
import numpy as np

plt.style.use('ggplot')

# Sentiment color scheme and stack order (reused across charts)
colors = {
    'negative': '#e74c3c',  # Red
    'neutral':  '#95a5a6',  # Gray
    'positive': '#2ecc71'   # Green
}
sentiment_order = ['negative', 'neutral', 'positive']
```

Aggregate by entity and sentiment, then identify entities that have *any* positive or negative sentiment (filtering out purely neutral entities).

Sort by negative mention count to surface the highest-risk entities first.

In [11]:
```
entity_sentiment = df.groupby(['ENTITY_TEXT', 'SENTIMENT'])['MENTIONS'].sum().reset_index()

# find entities that have positive OR negative sentiment (not just neutral)
entity_sentiment_set = df.groupby('ENTITY_TEXT')['SENTIMENT'].apply(lambda x: set(x))
                                                              # convert each groups sentiment values to a Python set

# filter to entities that have 'positive' or 'negative' in their sentiment set
entities_with_sentiment = entity_sentiment_set[
    entity_sentiment_set.apply(lambda x: 'positive' in x or 'negative' in x)
].index.tolist() # convert to list

print(f"Found {len(entities_with_sentiment)} entities with positive or negative sentiment")
```

```
Found 399 entities with positive or negative sentiment
```

In [12]:
```
# Get negative mention counts for these entities
negative_mentions = df[
    (df['ENTITY_TEXT'].isin(entities_with_sentiment)) &
    (df['SENTIMENT'] == 'negative')
].groupby('ENTITY_TEXT')['MENTIONS'].sum()
```

In [13]:
```
# sort by negative mentions (descending) and get top 10
all_sentiment_entities = pd.Series(0, index=entities_with_sentiment)
all_sentiment_entities.update(negative_mentions)
top_entities_by_negative = all_sentiment_entities.nlargest(25).index.tolist()
```

```
# filter to only include selected entities
entity_sentiment_top_negative = entity_sentiment[entity_sentiment['ENTITY_TEXT'].isin(top_entities_by_negative)]
```

Visualization

In [14]:
```
# Pivot the data so sentiments become columns (same logic as first viz)
pivot_entity_negative = entity_sentiment_top_negative.pivot(index='ENTITY_TEXT',
                                                            columns='SENTIMENT',
                                                            values='MENTIONS').fillna(0)

# Reorder columns to control the stack order
pivot_entity_negative = pivot_entity_negative.reindex(columns=sentiment_order, fill_value=0)

# Sort by NEGATIVE mentions (descending) - key difference from first visualization
pivot_entity_negative = pivot_entity_negative.sort_values('negative', ascending=False)
```

In [15]:
```
# create stacked bar chart (same visualization logic as first chart)
fig, ax = plt.subplots(figsize=(12, 6))

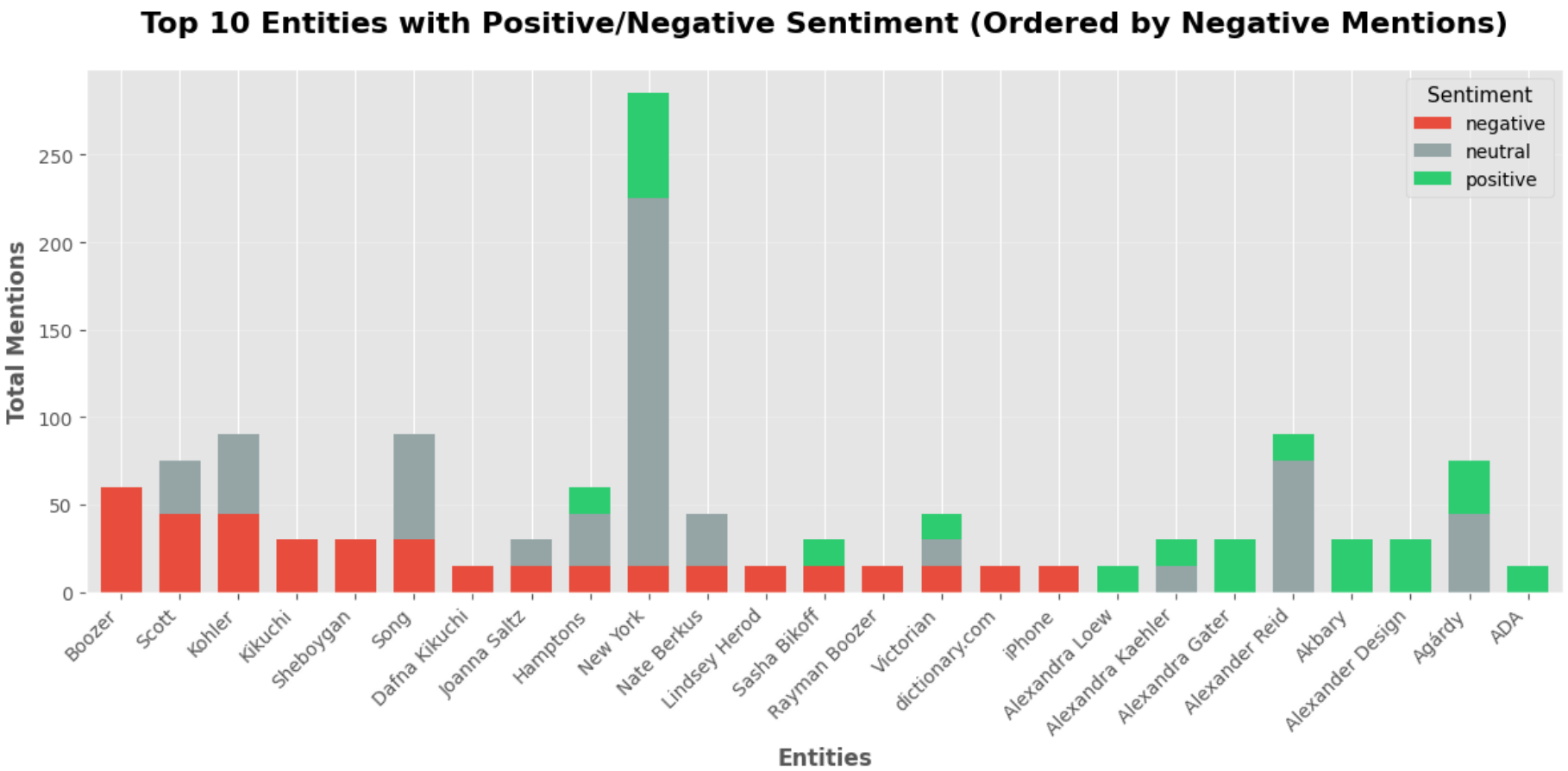
pivot_entity_negative.plot(kind='bar',
                           stacked=True,
                           ax=ax,
                           color=[colors[col] for col in pivot_entity_negative.columns],
                           width=0.7)

# Customize the chart
ax.set_title('Top 10 Entities with Positive/Negative Sentiment (Ordered by Negative Mentions)',
             fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('Entities', fontsize=12, fontweight='bold')
ax.set_ylabel('Total Mentions', fontsize=12, fontweight='bold')
ax.legend(title='Sentiment', title_fontsize=11, fontsize=10)
ax.set_xticklabels(pivot_entity_negative.index, rotation=45, ha='right')

# Add grid for better readability
ax.yaxis.grid(True, alpha=0.3)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()

print("\nNote: Entities are ordered left-to-right by negative mention count (highest first).")
```




```
Note: Entities are ordered left-to-right by negative mention count (highest first).
```

Observation

This graph brings it all together; on the farthest left side should be the entities with the highest (if not 100%) negative sentiment, decreasing to the right, where there are entities with more neutral and/or positive sentiment to the right. A client might use this graph to NOT ONLY determine out of all entities, which ones to avoid the most, but also what entities have the highest negative sentiment, while still having the *least bit* of positive (or neutral) sentiment.

This prioritization helps e-commerce clients quickly identify high-risk entities while understanding their full sentiment context. An entity with 100 negative mentions and 200 positive mentions tells a different story than one with 100 negative mentions and 0 positive mentions - both are shown here, but ordered by the negative signal strength.

Analyzing Confidence Scores for Negative Sentiment

WigglNET uses RoBERTa, which provides confidence scores (0–1) indicating how certain the model is about each sentiment classification. Examining confidence alongside negative mention counts distinguishes reliable warning signals from potentially misclassified or ambiguous entities.

The same approach can be applied to positive sentiment to surface partnership candidates, but is omitted here for brevity — the methodology is identical with the color map and sort field swapped.

Reusable Visualization Functions

In [16]:
```
import matplotlib.cm as cm
from matplotlib.colors import Normalize

def plot_confidence_bar_chart(data, mentions_col, confidence_col,
                              sentiment_type, color_map='Reds'):
    # sort by mentions for better visualization
    data_sorted = data.sort_values(mentions_col, ascending=True)

    # create figure object and ax
    fig, ax = plt.subplots(figsize=(12, 10))

    # normalize the confidence scores across the data,
    #   so that it can be color mapped
    norm = Normalize(vmin=data_sorted[confidence_col].min(),
                     vmax=data_sorted[confidence_col].max())
    cmap = cm.get_cmap(color_map) # call in color map

    # Create bars with color based on confidence
    ax.barh(range(len(data_sorted)),
            data_sorted[mentions_col],
            color=cmap(norm(data_sorted[confidence_col])))

    # Customize chart
    ax.set_yticks(range(len(data_sorted)))
    ax.set_yticklabels(data_sorted['ENTITY_TEXT'], fontsize=10)
    ax.set_xlabel(f'Total {sentiment_type} Mentions', fontsize=12, fontweight='bold')
    ax.set_ylabel('Entity', fontsize=12, fontweight='bold')
    ax.set_title(f'Top 25 {sentiment_type}ly Mentioned Entities\n(Color Intensity = Confidence Level)',
                 fontsize=14, fontweight='bold', pad=20)

    # Add grid
    ax.xaxis.grid(True, alpha=0.3)
    ax.set_axisbelow(True)

    # Add colorbar
    sm = cm.ScalarMappable(cmap=cmap, norm=norm)
    sm.set_array([])
    cbar = plt.colorbar(sm, ax=ax)
    cbar.set_label('Average Confidence Score', fontsize=11, fontweight='bold')

    # Add value labels on bars
    for i, (mentions, confidence) in enumerate(zip(data_sorted[mentions_col],
                                                   data_sorted[confidence_col])):
        ax.text(mentions + 1, i, f'{int(mentions)} ({confidence:.2f})',
                va='center', fontsize=8, color='black')

    plt.tight_layout()
    plt.show()
```

In [17]:
```
def plot_confidence_scatter(data, mentions_col, confidence_col,
                            sentiment_type, color_map='Reds',
                            quadrant_color_high='#ffcccc', quadrant_color_low='#ffffcc',
                            max_mentions=None):
    # Create figure
    fig, ax = plt.subplots(figsize=(14, 8))

    # Plot scatter points
    scatter = ax.scatter(data[mentions_col],
                         data[confidence_col],
                         s=200,
                         c=data[confidence_col],
                         cmap=color_map,
                         alpha=0.6,
                         edgecolors='black',
                         linewidth=1.5)

    # Add entity labels
    for idx, row in data.iterrows():
        ax.annotate(row['ENTITY_TEXT'],
                    (row[mentions_col], row[confidence_col]),
                    fontsize=9,
                    xytext=(5, 5),
                    textcoords='offset points',
                    bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow', alpha=0.3))

    # Add quadrant lines (median splits)
```

```
    median_mentions = data[mentions_col].median()
    median_confidence = data[confidence_col].median()

    ax.axvline(median_mentions, color='gray', linestyle='--', alpha=0.5, linewidth=1)
    ax.axhline(median_confidence, color='gray', linestyle='--', alpha=0.5, linewidth=1)

    # Add quadrant labels
    ax.text(median_mentions * 1.5, data[confidence_col].max() * 0.98,
            'High Confidence\nHigh Volume', ha='center', fontsize=10,
            bbox=dict(boxstyle='round', facecolor=quadrant_color_high, alpha=0.7))
    ax.text(median_mentions * 0.5, data[confidence_col].max() * 0.98,
            'High Confidence\nLow Volume', ha='center', fontsize=10,
            bbox=dict(boxstyle='round', facecolor=quadrant_color_low, alpha=0.7))

    # Customize chart
    ax.set_xlabel(f'Total {sentiment_type} Mentions', fontsize=12, fontweight='bold')
    ax.set_ylabel('Average Confidence Score', fontsize=12, fontweight='bold')
    ax.set_title(f'Confidence vs. {sentiment_type} Mention Count for Top 25 Entities',
                 fontsize=14, fontweight='bold', pad=20)
    ax.grid(True, alpha=0.3)

    # Cap the x-axis if max_mentions is specified
    if max_mentions is not None:
        ax.set_xlim(0, max_mentions)

    # Add colorbar
    cbar = plt.colorbar(scatter, ax=ax)
    cbar.set_label('Confidence Score', fontsize=11, fontweight='bold')

    plt.tight_layout()
    plt.show()
```

Data Preparation

In [18]:
```
# Filter for negative sentiment only
df_negative = df[df['SENTIMENT'] == 'negative'].copy()

print(f"Found {len(df_negative)} negative sentiment records")
print(f"Confidence score range: {df_negative['CONFIDENCE'].min():.3f} to {df_negative['CONFIDENCE'].max():.3f}")
print(f"Average confidence: {df_negative['CONFIDENCE'].mean():.3f}")
```

```
Found 255 negative sentiment records
Confidence score range: 0.519 to 0.825
Average confidence: 0.674
```

In [19]:
```
# Aggregate negative entities: sum mentions and average confidence
negative_entity_stats = df_negative.groupby('ENTITY_TEXT').agg({
    'MENTIONS': 'sum',
    'CONFIDENCE': 'mean'
}).reset_index()

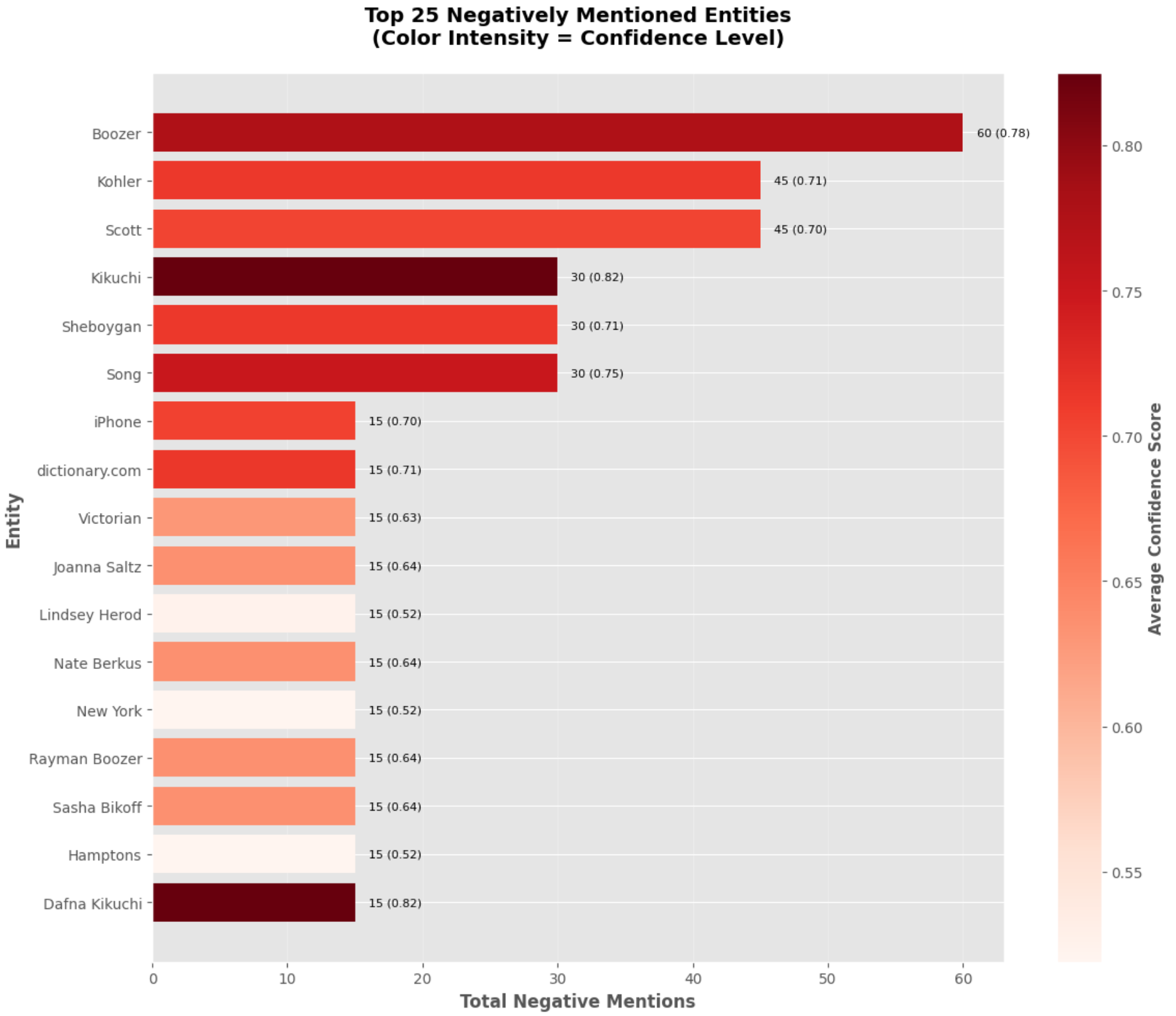
negative_entity_stats.columns = ['ENTITY_TEXT', 'TOTAL_NEGATIVE_MENTIONS', 'AVG_CONFIDENCE']
top_25_negative = negative_entity_stats.nlargest(25, 'TOTAL_NEGATIVE_MENTIONS')
top_25_negative['AVG_CONFIDENCE'] = top_25_negative['AVG_CONFIDENCE'].round(3)
```

Bar Chart: Color Gradient by Confidence

Bar height represents negative mention count, while color intensity shows confidence level. Darker bars indicate higher model confidence in the negative sentiment classification.

In [20]:
```
# Use the reusable function for negative entities bar chart
plot_confidence_bar_chart(
    data=top_25_negative,
    mentions_col='TOTAL_NEGATIVE_MENTIONS',
    confidence_col='AVG_CONFIDENCE',
    sentiment_type='Negative',
    color_map='Reds'
)
```

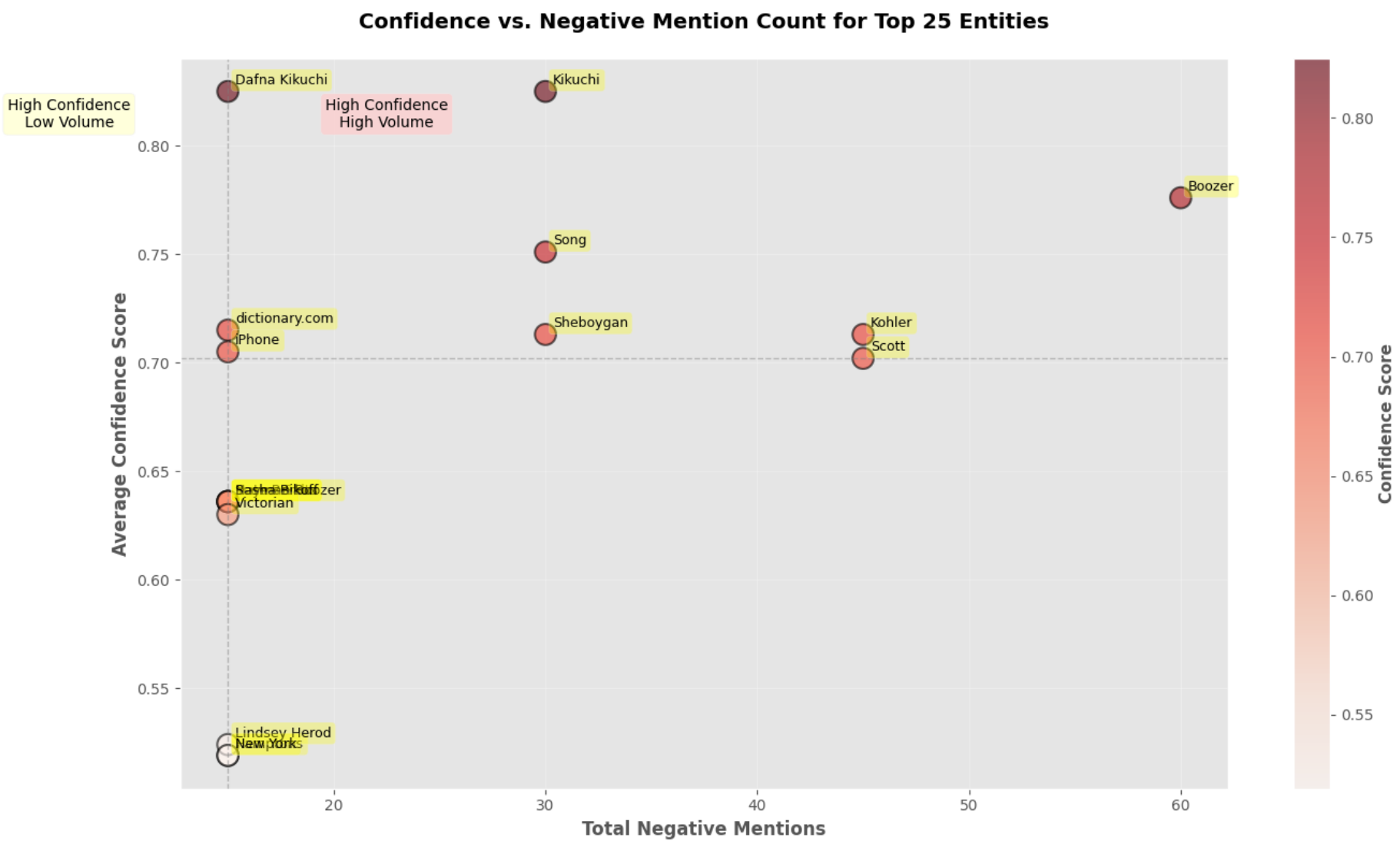
```
/tmp/ipykernel_71345/2958685450.py:16: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplot
lib 3.7 and will be removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or ``
pyplot.get_cmap()`` instead.
  cmap = cm.get_cmap(color_map) # call in color map
```



Scatter Plot: Confidence vs. Negative Mentions

The top-right quadrant (high mentions, high confidence) represents the most reliable warning signs. High-confidence/low-mention entities are emerging concerns, high-mention/low-confidence entities may be misclassifications worth investigating, and low/low entities are the least concerning.

In [21]:
```
# Use the reusable function for negative entities scatter plot
plot_confidence_scatter(
    data=top_25_negative,
    mentions_col='TOTAL_NEGATIVE_MENTIONS',
    confidence_col='AVG_CONFIDENCE',
    sentiment_type='Negative',
    color_map='Reds',
    quadrant_color_high='#ffcccc',
    quadrant_color_low='#ffffcc'
)
```



As mentioned before, entities on the farthest right side of the scatter plot have higher mentions, but those in the upper portion of the right side have high confidence scores.

These are what the different confidence thresholds mean.

(>0.85) - the RoBERTa model is very certain about the sentiment classification
(0.70-0.85) - the model is moderately certain, which is typical for clear negative language
(<0.70) - low confidence: uncertain classification. May indicate ambiguous language or misclassification

For WigglNet clients, high confidence, high volume entities will show up as darker, taller bars which indicate the most reliable negative signals. E-commerce sellers should investigate these entities thoroughly before associating with them.

This two-dimensional analysis helps clients distinguish between solid negative signals and potentially uncertain classifications, leading to better-informed business decisions.

Milestone 1 Conclusion

The best way for a seller or manufacturer to understand what's happening in a market is to look directly at what is being talked about. Reading articles one by one and making judgements for yourself whether an entity is positive or negative is an inefficient approach compared to the classification models WigglNet runs to identify the sentiment of keywords and entities across hundreds of articles. This is not to say though, that manually applying sentiment labels is useless. In order to develop "ground-truths" about the clients particular needs for the market, there must be human-made truths that the model can compare its predictions to. From there the model can improve its predictions based on the outcomes, results and KPIs of the client's analytics.

This notebook reviewed a process of validating results produced by RoBERTa to verify the effectiveness of the model's labelling for a particular target market. It was visualized in three stacked bar charts showing the different ways clients could use this data to make decisions about certain topics between positive and negative sentiment.

Milestone 2: Data Preparation

Next in the pipeline is preparing the data for machine learning model building and evaluation. In the following sections the data will undergo cleaning, extraction, selection, transformation, and feature engineering.

Drop any features that are not useful for your model building and explain why they are not useful.

In [22]:
```
df = df.drop(columns=["ARTICLE_ID", "CONFIDENCE"])
```

For model development, the only columns needed to be dropped is the `ARTICLE_ID` and `CONFIDENCE` field. Article ID is set automatically by the program creating the data and Confidence is RoBERTa's confidence in the sentiment label it produced. Feeding this into a classifier validating the accuracy of the labels, the model will look extremely accurate but will be under false pretenses, since the feature it's leaning on is derived from the very label it's trying to validate.

Check for missing data

In [23]:
```
# Sum the NaNs
df.isnull().sum()
```

Out[23]:
```
TITLE             0
PUBLISHED_DATE    0
KEYWORD_TEXT      0
ENTITY_TEXT       0
ENTITY_TYPE       0
SENTIMENT         0
MENTIONS          0
dtype: int64
```

A check using `df.isnull().sum()` returned zero null values across all columns. This is expected: the data was queried from a database populated by a clean upstream pipeline, and the SQL query explicitly filtered out rows where PUBLISHED_DATE IS NOT NULL. No imputation or row removal was required.

Engineering new features: Published Date

From the `PUBLISHED_DATE` column four features will be created:

`PUB_YEAR` : Year the article was published
`PUB_MONTH` : Month the article was published
`PUB_DOW` : Day of the week article was published

In [24]:
```
# converting to datetime if not already
df['PUBLISHED_DATE'] = pd.to_datetime(df['PUBLISHED_DATE'])

# adding the three new features
df['pub_year']       = df['PUBLISHED_DATE'].dt.year
df['pub_month']      = df['PUBLISHED_DATE'].dt.month
df['pub_dow']        = df['PUBLISHED_DATE'].dt.dayofweek   # 0=Monday, 6=Sunday

# clean up by dropping PUBLISHED_DATE
df = df.drop(columns=['PUBLISHED_DATE'])
```

Feature transformations: Mentions

In [25]:
```
# Imports and preprocessing
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import OneHotEncoder
import category_encoders as ce
```

`MENTIONS` is skewed in that a few entities are mentioned hundreds of time. Applying log compresses the range and helps linear models. Below is the current histogram of the mentions field - it can be concluded that this graph is right-skewed.

In [26]:
```
# pick a representative spread of values from your data
# Sample representative values from the actual distribution
sample_values = np.unique(
    np.percentile(df['MENTIONS'], [10, 25, 50, 75, 90, 95, 100]).astype(int)
)
log_values = np.log1p(sample_values)

fig, axes = plt.subplots(2, 1, figsize=(14, 5))

# === Top: raw MENTIONS number line ===
axes[0].hlines(0, 0, 40, colors='black', linewidth=1)
axes[0].scatter(sample_values, [0]*len(sample_values),
                color='#e74c3c', s=120, zorder=3, edgecolor='black')

# Label each point with its value
for v in sample_values:
    axes[0].annotate(str(v), (v, 0), textcoords='offset points',
                     xytext=(0, 12), ha='center', fontsize=10)

axes[0].set_xlim(-1, 41)
axes[0].set_ylim(-1, 1)
axes[0].set_title('Before: Raw MENTIONS', fontsize=13, fontweight='bold', loc='left')
axes[0].set_yticks([])
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)
axes[0].spines['left'].set_visible(False)

# === Bottom: log-transformed number line ===
axes[1].hlines(0, 0, 4, colors='black', linewidth=1)
axes[1].scatter(log_values, [0]*len(log_values),
                color='#2ecc71', s=120, zorder=3, edgecolor='black')

for raw, log_v in zip(sample_values, log_values):
    axes[1].annotate(f'{log_v:.2f}', (log_v, 0), textcoords='offset points',
                     xytext=(0, 12), ha='center', fontsize=10)

axes[1].set_xlim(-0.1, 4.1)
```

```
axes[1].set_ylim(-1, 1)
axes[1].set_title('After: log1p(MENTIONS)', fontsize=13, fontweight='bold', loc='left')
axes[1].set_yticks([])
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)
axes[1].spines['left'].set_visible(False)

plt.suptitle('Redistribute Spacing of MENTIONS',
             fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



Creating a before/after of applying the logarithm it looks as if it did not change much. However what it did was compress the long right tail.

In the raw data, a row with 38 mentions sat 37 units away from a row with 1 mention, a distance the model would treat as enormous. After log1p, that same gap shrinks to about 3 units, putting high- and low-mention rows on a comparable scale. The visual shape doesn't change dramatically because most rows already cluster at 1–3 mentions, but the underlying distance relationships between values are now proportional, which is what linear models actually need.

Feature Encoding: Text and Keywords

`ENTITY_TEXT` and `KEYWORD_TEXT` cannot be read by a machine learning model as the text themselves. A method called TargetEncoding must be applied to convert them to a numeric value that represents how closely the keyword is to a positive/negative. A value closer to 0 means the entity is usually tagged negative, while a value near 2 means it's usually tagged positive.

In [27]:
```
encoder = ce.TargetEncoder(cols=['ENTITY_TEXT', 'KEYWORD_TEXT'])

encoded = encoder.fit_transform(
    df[['ENTITY_TEXT', 'KEYWORD_TEXT']],
    df['SENTIMENT']
)

# Rename so it's clear these are encoded versions
encoded.columns = ['ENTITY_ENC', 'KEYWORD_ENC']

# Drop originals, merge encoded versions back in
df = pd.concat([df.drop(columns=['ENTITY_TEXT', 'KEYWORD_TEXT']), encoded], axis=1)

print("After target encoding:")
print(df[['ENTITY_ENC', 'KEYWORD_ENC']].describe())
```

```
After target encoding:
         ENTITY_ENC   KEYWORD_ENC
count  19367.000000  19367.000000
mean       1.355152      1.359714
std        0.238966      0.062824
min        0.731219      1.128517
25%        1.223921      1.333507
50%        1.223921      1.355606
75%        1.601461      1.389829
max        2.000000      1.579529
```

Next, TfidfVectorizer is used to encode the TITLE feature. Stop words will drop common words like "the", "of", "is", "and" that appear in every title and carry no signal.

In [28]:
```
tfidf = TfidfVectorizer(
    max_features=100,
    stop_words='english',
    ngram_range=(1, 2)
)

title_matrix = tfidf.fit_transform(df['TITLE'])

title_df = pd.DataFrame(
```

```
    title_matrix.toarray(),
    columns=[f'title_{w}' for w in tfidf.get_feature_names_out()],
    index=df.index
)

df = pd.concat([df.drop(columns=['TITLE']), title_df], axis=1)

print(f"After TF-IDF, added {title_df.shape[1]} title-derived columns")
print("Sample title features:", title_df.columns[:10].tolist())
```

```
After TF-IDF, added 100 title-derived columns
Sample title features: ['title_2026', 'title_2026 color', 'title_45', 'title_alyssa', 'title_alyssa khan', 'title_an
nounces', 'title_announces 2026', 'title_archive', 'title_bathroom', 'title_bathroom vanity']
```

One hundred new parameters derived from `TITLE` have just been added to the dataframe. Each column corresponds to one word or two-word phrase from the title vocabulary, weighted by TF-IDF, a score that rewards terms appearing frequently in a given title while penalizing terms that appear in every title. The result is a numeric representation of the title text that the model can use as features, with common filler words ("the", "of") filtered out and only the 100 most informative terms retained.

One Hot Encoding: Entity Type

In [29]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
import category_encoders as ce
```

`ENTITY_TYPE` has five unique values. (PERSON, ORG, GPE, PRODUCT). Since there are no meaningful rankings, one hot encoding could be used to convert them to numeric values that can be read by a model.

One hot encoding will produce "dummy variables", or placeholders for categorical information. They flip 1 or 0 depending on which category a row belongs to. It is a "dummy" variable because it doesn't measure a quantity, but indicates membership in a group.

In [30]:
```
entity_type_dummies = pd.get_dummies(df['ENTITY_TYPE'], prefix='etype', dtype=int)
df = pd.concat([df.drop(columns=['ENTITY_TYPE']), entity_type_dummies], axis=1)

print("After one-hot encoding ENTITY_TYPE:")
print(df.filter(like='etype_').head())
```

```
After one-hot encoding ENTITY_TYPE:
   etype_GPE  etype_NORP  etype_ORG  etype_PERSON  etype_PRODUCT
0          0           0          1             0              0
1          0           0          1             0              0
2          0           0          0             1              0
3          0           0          0             1              0
4          0           1          0             0              0
```

Reviewing the new dataframe, each predictor is now in a numeric form that can be processed by a model.

Ready for Modeling

All feature columns are now numeric and encoded. `SENTIMENT` (the target) will be separated before training; every remaining column is in a format the model can consume directly.

In [36]:
```
df.dtypes.value_counts()
```

Out[36]:
```
float64    102
int64        6
int32        3
str          1
Name: count, dtype: int64
```

Milestone 2 Conclusion

In Milestone 2, each categorical field was converted to a numeric field or set of fields that represented the field it was derived from. The benefit of data that can be understood by humans is that it's easily communicable. Teams speaking the same language can understand insights about the data simply by reading and passing along. ML models, however, cannot understand data in our language.

In cases like the Mentions transformation, log was needed to redistribute the spacing between values so the model could interpret them on a proportional scale rather than being dominated by a few outliers. For the text and keywords, Target Encoding was applied because one-hot encoding 893 entities and 655 keywords would have created over 1,500 sparse columns. Instead, each category is replaced with a single float representing its average sentiment, compressing the signal into one column per field.

Other data preparation steps included feature engineering on PUBLISHED_DATE to extract year, month, day-of-week, and days-since-publication; dropping ARTICLE_ID and CONFIDENCE (the former an auto-generated identifier with no predictive value, the latter a source of target leakage); one-hot encoding ENTITY_TYPE into five dummy variables; and TF-IDF vectorizing TITLE into 100 weighted term features. A check for missing values returned none, owing to the clean upstream pipeline that populated the source database.